



YAPAY ZEKA MÜHENDİSLİĞİ BÖLÜMÜ

DERİN ÖĞRENME DERSİ PROJE RAPORU

Öğrenci: Riyam Albayati

Öğrenci No: 230212905

Ders: Derin Öğrenme YZM (306)

Ders Sorumlusu: Dr. Öğr. Üyesi Murat Şimşek

2025-2026 Bahar Dönemi

YOLOv8 Tabanlı No-Code Model Eğitimi ve Nesne Tespiti Sistemleri Geliştirilmesi

1. GİRİŞ

Yapay zeka teknolojilerinin gelişmesiyle birlikte görüntü işleme ve nesne tespiti sistemleri birçok alanda aktif olarak kullanılmaya başlanmıştır. Özellikle derin öğrenme tabanlı nesne tespiti modelleri; güvenlik sistemleri, sağlık teknolojileri, otonom araçlar, üretim otomasyonu ve akıllı şehir uygulamalarında önemli bir yere sahiptir.

Bu projede, kullanıcıların teknik bilgiye ihtiyaç duymadan YOLOv8 modeli eğitebilmesini ve eğitilmiş modeli kullanarak nesne tespiti gerçekleştirebilmesini sağlayan iki ayrı grafiksel kullanıcı arayüzü (GUI) geliştirilmiştir.

Geliştirilen sistemler:

- No-Code YOLOv8 Trainer GUI
- YOLO Object Detection GUI

olarak iki ana bölümden oluşmaktadır.

Projede Python programlama dili, Ultralytics YOLOv8 kütüphanesi ve Gradio web arayüzü teknolojileri kullanılmıştır.

2. PROJENİN AMACI

Bu projenin temel amacı, yapay zeka tabanlı nesne tespiti süreçlerini kullanıcı dostu hale getirmek ve terminal komutlarına ihtiyaç duymadan model eğitimi ile inference işlemlerinin gerçekleştirilebilmesini sağlamaktır.

Projede geliştirilen sistemler sayesinde:

- Kullanıcılar yalnızca veri seti yükleyerek model eğitebilmektedir.
- Eğitilen modeller GUI üzerinden kullanılabilir.
- Nesne tespiti sonuçları görsel olarak analiz edilebilmektedir.
- Teknik bilgi gereksinimi azaltılmıştır.

- YOLOv8 eğitim ve inference süreçleri otomatikleştirilmiştir.

3. KULLANILAN TEKNOLOJİLER

Projede aşağıdaki teknolojiler kullanılmıştır:

Teknoloji	Açıklama
Python	Ana programlama dili
YOLOv8	Nesne tespiti modeli
Ultralytics	YOLOv8 kütüphanesi
Gradio	Web tabanlı GUI geliştirme
OpenCV	Görüntü işleme işlemleri
NumPy	Sayısal veri işlemleri
Pillow (PIL)	Görsel format dönüşümleri
Roboflow	Veri seti yönetimi
Google Colab	GPU destekli eğitim ortamı

4. YOLOv8 HAKKINDA TEORİK BİLGİ

YOLO (You Only Look Once), gerçek zamanlı nesne tespiti için geliştirilmiş derin öğrenme tabanlı bir algoritmadır.

YOLOv8 modeli:

- Tek aşamalı object detection mimarisi kullanır.
- Yüksek hız ve doğruluk sağlar.
- Gerçek zamanlı inference işlemleri gerçekleştirebilir.
- Bounding box ve sınıf tahminlerini aynı anda üretir.

Bu projede YOLOv8m modeli kullanılmıştır.

5. PROJE 1 — NO-CODE YOLOv8 TRAINER GUI

5.1 Proje Tanımı

Bu proje, kullanıcıların herhangi bir terminal komutu yazmadan yalnızca bir ZIP veri seti yükleyerek YOLOv8 modeli eğitebilmesini sağlayan web tabanlı bir arayüzdür.

Sistem özellikle teknik bilgi seviyesi düşük kullanıcıların yapay zeka modeli eğitmesini kolaylaştırmak amacıyla geliştirilmiştir.

5.2 Sistem Çalışma Mantığı

Sistem aşağıdaki adımlarla çalışmaktadır:

1. Kullanıcı ZIP formatında veri seti yükler.
2. Sistem ZIP dosyasını açar.
3. Dataset formatı analiz edilir.
4. data.yaml dosyası kontrol edilir.
5. train / valid klasör yapısı doğrulanır.
6. Eğitim parametreleri alınır.
7. YOLOv8 modeli başlatılır.
8. Model eğitimi gerçekleştirilir.
9. best.pt modeli oluşturulur.
10. Eğitilen model kullanıcıya sunulur.

5.3 Veri Seti Kontrol Mekanizması

Sistem veri setinin doğru formatta olup olmadığını otomatik olarak kontrol etmektedir.

Kontrol edilen yapılar:

- data.yaml

- train/images
- train/labels
- valid/images
- valid/labels

Ayrıca sistem:

- COCO formatını
- VOC formatını

tespit ederek kullanıcıyı uyarmaktadır.

Bu sayede hatalı veri seti kullanımının önüne geçilmektedir.

5.4 Kullanılan Parametreler

Parametre	Açıklama
Epoch	Eğitim tekrar sayısı
Batch Size	Aynı anda işlenen veri miktarı
Image Size (imgsz)	Görsellerin yeniden boyutlandırılması
Model	YOLOv8m.pt

5.5 Model Eğitimi

Model eğitimi aşağıdaki komut ile gerçekleştirilmiştir:

```
yolo train model=yolov8m.pt data=data.yaml epochs=45 imgsz=640 batch=16
```

Bu eğitim sürecinde:

- GPU hızlandırması kullanılmıştır.
- Eğitim kayıpları izlenmiştir.
- Overfitting analizleri yapılmıştır.
- Precision, Recall ve mAP değerleri değerlendirilmiştir.

5.6 Arayüz Özellikleri

Geliştirilen GUI aşağıdaki özelliklere sahiptir:

- ZIP dataset yükleme
- Parametre belirleme
- Otomatik dataset doğrulama
- Hata mesajı sistemi
- Eğitilen modeli indirme
- Kullanıcı dostu tasarım

5.7 Karşılaşılan Problemler ve Çözümler

Problem 1: Hatalı Dataset Yapısı

Bazı veri setlerinde train/valid klasör yapısı eksikti.

Çözüm:

Sistem içerisine otomatik klasör doğrulama mekanizması eklendi.

Problem 2: Desteklenmeyen Veri Setleri

COCO ve VOC formatındaki veri setleri doğrudan eğitilememekteydi.

Çözüm:

Format kontrol sistemi geliştirilerek kullanıcıya uyarı mesajları gösterildi.

Problem 3: Eğitim Sırasında Hatalar

Eksik dosya veya bozuk dataset nedeniyle eğitim durabiliyordu.

Çözüm:

Try-except hata yönetim sistemi kullanıldı.

6. PROJE 2 — YOLO OBJECT DETECTION GUI

6.1 Proje Tanımı

Bu proje, eğitilmiş YOLOv8 modellerinin görseller üzerinde nesne tespiti yapmasını sağlayan web tabanlı bir GUI sistemidir.

Kullanıcı yalnızca:

- best.pt modelini
- görüntüyü

yükleyerek nesne tespiti gerçekleştirebilmektedir.

6.2 Sistem Çalışma Mantığı

Sistem aşağıdaki adımlarla çalışmaktadır:

1. Kullanıcı eğitilmiş modeli yükler.
2. Kullanıcı görüntü yükler.
3. Görüntü NumPy array formatına dönüştürülür.
4. YOLOv8 inference işlemi çalıştırılır.
5. Bounding box çizilir.
6. Nesne bilgileri çıkarılır.
7. Sonuç görselleştirilir.
8. Çıktı kullanıcıya sunulur.

6.3 Inference Süreci

Inference işlemi aşağıdaki parametrelerle gerçekleştirilmiştir:

Parametre	Açıklama
Confidence Threshold	Minimum güven skoru
IoU Threshold	Bounding box çakışma oranı
Selected Class	Filtrelenecek sınıf

6.4 Confidence ve IoU Kavramları

Confidence Threshold

Modelin yaptığı tahminin güven skorudur.

Örneğin:

$$0 \leq \textit{Confidence} \leq 1$$

Confidence değeri yükseldikçe yanlış pozitif sonuçlar azalır.

IoU (Intersection over Union)

Bounding box çakışma oranını ifade eder.

$$IoU = \frac{\textit{Area of Overlap}}{\textit{Area of Union}}$$

Bu değer overlapping kutuların filtrelenmesinde kullanılır.

6.5 Sistem Özellikleri

Geliştirilen GUI aşağıdaki özellikleri içermektedir:

- Model yükleme
- Görüntü yükleme
- Webcam desteği
- Confidence ayarı
- IoU ayarı
- Class filtreleme
- FPS hesaplama
- Bounding box çizimi
- Detection count sistemi
- Sonuç indirme

6.6 FPS Hesaplama

Sistemde gerçek zamanlı performans analizi için FPS ölçümü yapılmıştır.

FPS hesaplaması:

$$FPS = \frac{1}{Time\ Per\ Frame}$$

Bu sayede modelin inference performansı ölçülmüştür.

6.7 Bounding Box İşlemleri

Tespit edilen nesnelerin koordinatları alınarak görüntü üzerine kutular çizilmiştir.

Kullanılan bilgiler:

- x1
- y1
- x2
- y2
- class id
- confidence score

Bu bilgiler kullanıcıya detaylı şekilde gösterilmiştir.

6.8 Hata Yönetimi

Sistemde aşağıdaki durumlar kontrol edilmiştir:

- Model yüklenmemesi
- Görüntü yüklenmemesi
- Geçersiz dosya formatı
- Inference hataları

Hata durumlarında kullanıcıya açıklayıcı mesajlar gösterilmiştir.

7. PROJELERİN BİRLİKTE DEĞERLENDİRİLMESİ

Geliştirilen iki proje birlikte değerlendirildiğinde tam bir yapay zeka pipeline'ı oluşturulmuştur.

Süreç aşağıdaki gibidir:

1. Veri seti hazırlanır.
2. Model eğitilir.
3. Eğitilen model kaydedilir.
4. Model inference sistemine yüklenir.
5. Nesne tespiti gerçekleştirilir.
6. Sonuçlar analiz edilir.

Bu yapı gerçek dünya yapay zeka projelerine benzer bir çalışma sistemi sunmaktadır.

8. TEST VE PERFORMANS ANALİZİ

Projeler farklı veri setleri üzerinde test edilmiştir.

Yapılan testler:

- Dataset doğrulama testleri
- Eğitim parametre testleri
- Inference doğruluk testleri
- GUI kullanım testleri
- Hata senaryosu testleri

Analiz edilen metrikler:

- Precision
- Recall
- mAP
- FPS
- Detection Count

9. PROJENİN AVANTAJLARI

Geliştirilen sistemlerin sağladığı avantajlar:

- Teknik bilgi ihtiyacını azaltması
- Kullanıcı dostu arayüz sunması
- Eğitim ve inference süreçlerini otomatikleştirmesi
- Hata kontrol mekanizmaları içermesi
- Web tabanlı çalışabilmesi
- Gerçek zamanlı nesne tespiti yapabilmesi

10. GELECEKTE YAPILABİLECEK GELİŞTİRMELER

Projeye gelecekte aşağıdaki geliştirmeler eklenebilir:

- Video desteği
- Gerçek zamanlı kamera takibi

- Çoklu model desteđi
- Segmentasyon desteđi
- TensorRT optimizasyonu
- Bulut tabanlı eđitim sistemi
- Kullanıcı hesap sistemi
- Eđitim grafikleri gösterimi
- Model karşılařtırma sistemi

11. SONUÇ

Bu projede YOLOv8 tabanlı iki farklı GUI sistemi geliştirilmiştir.

Birinci sistem ile kullanıcıların teknik bilgi gerektirmeden model eğitmesi sağlanmıştır.

İkinci sistem ile eğitilen modeller kullanılarak nesne tespiti yapılmıştır.

Projeler kapsamında:

- Derin öğrenme
- Görüntü işleme
- Object detection
- GUI geliştirme
- Veri seti yönetimi
- Model eğitimi
- Inference süreçleri

alanlarında uygulamalı çalışmalar gerçekleştirilmiştir.

Bu çalışmalar sonucunda kullanıcı dostu, modüler ve geliştirilebilir yapay zeka sistemleri başarıyla geliştirilmiştir.

YOLO Object Detection GUI

Eğitilmiş YOLO modelinizi (best.pt) ve bir görüntü yükleyerek nesne tespiti yapabilirsiniz. Çıktı olarak sınıflar, confidence skoru ve bounding box koordinatları gösterilir.

YOLO Model Dosyası (best.pt)

YOLO26n.pt

Yükle

Yükle

Confidence Threshold

0.5

0.10

IOU Threshold

0.5

0.45

Gösterilecek sınıf

All

Clear

Submit

Tespit sonucu (Bounding Box)

Tespit Detayları

1) Sınıf: bus | Confidence: 0.76 | Koordinatlar: (x_min=376, y_min=103, x_max=443, y_max=283)

2) Sınıf: car | Confidence: 0.74 | Koordinatlar: (x_min=412, y_min=272, x_max=482, y_max=315)

3) Sınıf: bus | Confidence: 0.69 | Koordinatlar: (x_min=535, y_min=150, x_max=579, y_max=205)

4) Sınıf: bus | Confidence: 0.50 | Koordinatlar: (x_min=287, y_min=152, x_max=310, y_max=187)

5) Sınıf: bus | Confidence: 0.34 | Koordinatlar: (x_min=200, y_min=255, x_max=340, y_max=318)

6) Sınıf: car | Confidence: 0.33 | Koordinatlar: (x_min=432, y_min=276, x_max=479, y_max=319)

7) Sınıf: car | Confidence: 0.31 | Koordinatlar: (x_min=222, y_min=258, x_max=258, y_max=288)

8) Sınıf: car | Confidence: 0.22 | Koordinatlar: (x_min=209, y_min=201, x_max=251, y_max=238)

9) Sınıf: car | Confidence: 0.21 | Koordinatlar: (x_min=328, y_min=226, x_max=358, y_max=256)

10) Sınıf: car | Confidence: 0.19 | Koordinatlar: (x_min=397, y_min=106, x_max=409, y_max=111)

11) Sınıf: bus | Confidence: 0.19 | Koordinatlar: (x_min=775, y_min=186, x_max=816, y_max=237)

12) Sınıf: bus | Confidence: 0.15 | Koordinatlar: (x_min=248, y_min=156, x_max=279, y_max=209)

13) Sınıf: truck | Confidence: 0.15 | Koordinatlar: (x_min=452, y_min=275, x_max=479, y_max=319)

14) Sınıf: traffic light | Confidence: 0.14 | Koordinatlar: (x_min=468, y_min=75, x_max=473, y_max=85)

15) Sınıf: truck | Confidence: 0.13 | Koordinatlar: (x_min=178, y_min=187, x_max=217, y_max=256)

16) Sınıf: bus | Confidence: 0.12 | Koordinatlar: (x_min=178, y_min=187, x_max=217, y_max=256)

17) Sınıf: bus | Confidence: 0.11 | Koordinatlar: (x_min=300, y_min=222, x_max=315, y_max=241)

18) Sınıf: car | Confidence: 0.10 | Koordinatlar: (x_min=412, y_min=188, x_max=433, y_max=212)

Inference süresi (fps)

13.81 FPS

Toplam Tespit

16

Çıktıyı indir

annotated_output.jpg

45.1 KB

Flag

No-Code YOLOv8 Trainer

YOLO formatında, images-labels klasörleri ve data.yaml içeren ZIP veri seti yükleyiniz. Eksik dosya olması durumunda sistem sizi uyarır. Eğitim tamamlandığında eğitilen model dosyasını indirebilirsiniz.

YOLO Veri Seti (ZIP)

yolov8-toolsDetection.v7i.yolov8.zip

22.3 MB

Epoch

35

Batch Size

16

Image Size

640

Clear

Submit

Durum

Eğitim Tamamlandı! Model: /content/runs/detect/custom_yolo_model/weights/best.pt

Eğitilen Model (best.pt)

best.pt

49.6 MB

Flag

Use via API · Built with Gradio · Settings

12. KAYNAKÇA

1. Ultralytics YOLOv8 Documentation
2. Python Official Documentation
3. Gradio Documentation
4. OpenCV Documentation
5. Roboflow Documentation
6. Google Colab Documentation
7. Deep Learning for Computer Vision kaynakları